

VELORA — Module Sales (Part 1)

Migrations · Enums · Models

Folder Structure

```
app/Modules/Sales/  
├── Controllers/  
│   ├── PaymentMethodController.php  
│   ├── ShiftController.php  
│   └── TransactionController.php  
├── DTOs/  
│   ├── CreateShiftDTO.php  
│   ├── CloseShiftDTO.php  
│   └── CreateTransactionDTO.php  
├── Enums/  
│   ├── ShiftStatus.php  
│   ├── TransactionStatus.php  
│   └── PaymentType.php  
├── Models/  
│   ├── PaymentMethod.php  
│   ├── Shift.php  
│   ├── Transaction.php  
│   └── TransactionItem.php  
├── Repositories/  
│   ├── Contracts/  
│   │   ├── PaymentMethodRepositoryInterface.php  
│   │   ├── ShiftRepositoryInterface.php  
│   │   └── TransactionRepositoryInterface.php  
│   └── Eloquent/  
│       ├── PaymentMethodRepository.php  
│       ├── ShiftRepository.php  
│       └── TransactionRepository.php  
├── Requests/  
│   ├── StorePaymentMethodRequest.php  
│   ├── OpenShiftRequest.php  
│   ├── CloseShiftRequest.php  
│   └── StoreTransactionRequest.php  
├── Resources/  
│   ├── PaymentMethodResource.php  
│   ├── ShiftResource.php  
│   └── TransactionResource.php  
├── Services/  
│   ├── PaymentMethodService.php  
│   ├── ShiftService.php  
│   └── TransactionService.php
```

└─ Routes/
└─ api.php

1. Migrations

Payment Methods Table

```
<?php
// database/migrations/sales/2026_05_19_000040_create_payment_methods_table.php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('payment_methods', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->uuid('business_id');

            $table->string('name', 50);           // Cash, BCA, QRIS
            $table->string('type', 20);           // cash, transfer, ewallet, edc, qris
            $table->string('provider', 50)->nullable(); // midtrans, manual, etc
            $table->string('account_number', 50)->nullable();

            $table->boolean('is_active')->default(true);
            $table->integer('sort_order')->default(0);

            $table->timestamps();
            $table->softDeletes();

            $table->foreign('business_id')->references('id')->on('businesses')->cascadeOnDelete();
            $table->index(['business_id', 'is_active', 'sort_order']);
        });
    }

    public function down(): void { Schema::dropIfExists('payment_methods'); }
};
```

Shifts Table

```
<?php
```

```
// database/migrations/sales/2026_05_19_000041_create_shifts_table.php
```

```
use Illuminate\Database\Migrations\Migration;
```

```
use Illuminate\Database\Schema\Blueprint;
```

```
use Illuminate\Support\Facades\Schema;
```

```
return new class extends Migration
```

```
{
```

```
    public function up(): void
```

```
    {
```

```
        Schema::create('shifts', function (Blueprint $table) {
```

```
            $table->uuid('id')->primary();
```

```
            $table->uuid('business_id');
```

```
            $table->uuid('branch_id');
```

```
            $table->uuid('user_id')->comment('Cashier');
```

```
            $table->timestamp('opened_at');
```

```
            $table->timestamp('closed_at')->nullable();
```

```
            $table->bigInteger('starting_cash')->default(0);
```

```
            $table->bigInteger('actual_ending_cash')->default(0)->comment('Inputan fisik kasir');
```

```
            $table->bigInteger('expected_ending_cash')->default(0)->comment('Hitungan sistem');
```

```
            $table->bigInteger('difference')->default(0)->comment('Selisih (actual - expected)');
```

```
            $table->string('status', 20)->default('open')->comment('open, closed');
```

```
            $table->text('notes')->nullable();
```

```
            $table->timestamps();
```

```
            $table->foreign('business_id')->references('id')->on('businesses')->cascadeOnDelete();
```

```
            $table->foreign('branch_id')->references('id')->on('branches');
```

```
            $table->foreign('user_id')->references('id')->on('users');
```

```
            $table->index(['business_id', 'branch_id', 'status']);
```

```
            $table->index(['user_id', 'status']);
```

```
        });
```

```
    }
```

```
public function down(): void { Schema::dropIfExists('shifts'); }  
};
```

Transactions Table

```
<?php
```

```
// database/migrations/sales/2026_05_19_000042_create_transactions_table.php
```

```
use Illuminate\Database\Migrations\Migration;
```

```
use Illuminate\Database\Schema\Blueprint;
```

```
use Illuminate\Support\Facades\Schema;
```

```
return new class extends Migration
```

```
{
```

```
    public function up(): void
```

```
    {
```

```
        Schema::create('transactions', function (Blueprint $table) {
```

```
            $table->uuid('id')->primary();
```

```
            $table->uuid('business_id');
```

```
            $table->uuid('branch_id');
```

```
            $table->uuid('user_id')->comment('Cashier / Staff');
```

```
            $table->uuid('shift_id')->nullable();
```

```
            $table->uuid('customer_id')->nullable(); // Dari Module CRM nanti
```

```
            $table->uuid('payment_method_id')->nullable();
```

```
            $table->string('invoice_number', 50)->unique();
```

```
            $table->string('status', 20)->default('completed')->comment('pending, completed, re
```

```
            $table->string('payment_status', 20)->default('paid')->comment('unpaid, paid, parti
```

```
            $table->string('order_type', 20)->default('dine_in')->comment('dine_in, take_away, c
```

```
            // Amount Details
```

```
            $table->bigInteger('subtotal')->default(0);
```

```
            $table->bigInteger('discount_amount')->default(0);
```

```
            $table->bigInteger('tax_amount')->default(0);
```

```
            $table->bigInteger('service_charge')->default(0);
```

```
            $table->bigInteger('grand_total')->default(0);
```

```
            $table->bigInteger('amount_paid')->default(0);
```

```
            $table->bigInteger('change_amount')->default(0);
```

```
            $table->string('currency', 3)->default('IDR');
```

```
            // Midtrans & EDC reference
```

```
            $table->string('payment_reference')->nullable();
```

```
            $table->text('notes')->nullable();
```

```
$table->timestamps();
```

```
$table->foreign('business_id')->references('id')->on('businesses')->cascadeOnDelete;
```

```
$table->foreign('branch_id')->references('id')->on('branches');
```

```
$table->foreign('user_id')->references('id')->on('users');
```

```
$table->foreign('shift_id')->references('id')->on('shifts');
```

```
$table->foreign('payment_method_id')->references('id')->on('payment_methods');
```

```
$table->index(['business_id', 'branch_id', 'created_at']);
```

```
$table->index(['status', 'payment_status']);
```

```
});
```

```
}
```

```
public function down(): void { Schema::dropIfExists('transactions'); }
```

```
};
```


Transaction Items Table

```
<?php
```

```
// database/migrations/sales/2026_05_19_000043_create_transaction_items_table.php
```

```
use Illuminate\Database\Migrations\Migration;
```

```
use Illuminate\Database\Schema\Blueprint;
```

```
use Illuminate\Support\Facades\Schema;
```

```
return new class extends Migration
```

```
{
```

```
    public function up(): void
```

```
    {
```

```
        Schema::create('transaction_items', function (Blueprint $table) {
```

```
            $table->uuid('id')->primary();
```

```
            $table->uuid('transaction_id');
```

```
            $table->uuid('product_id')->nullable(); // Nullable in case product deleted
```

```
            $table->uuid('product_variant_id')->nullable();
```

```
            // Snapshot of data during transaction
```

```
            $table->string('product_name', 200);
```

```
            $table->string('variant_name', 150)->nullable();
```

```
            $table->string('sku', 100)->nullable();
```

```
            $table->integer('quantity');
```

```
            $table->bigInteger('unit_price')->default(0);
```

```
            $table->bigInteger('cost_price')->default(0)->comment('For profit calculation');
```

```
            $table->bigInteger('discount_amount')->default(0);
```

```
            $table->bigInteger('tax_amount')->default(0);
```

```
            $table->bigInteger('subtotal')->default(0)->comment('qty * (unit_price - discount) -
```

```
            $table->text('notes')->nullable();
```

```
            $table->timestamps();
```

```
            $table->foreign('transaction_id')->references('id')->on('transactions')->cascadeOnDe
```

```
            $table->foreign('product_id')->references('id')->on('products')->nullOnDelete();
```

```
            $table->foreign('product_variant_id')->references('id')->on('product_variants')->nu
```

```
        });
```

```
    }
```

```
        public function down(): void { Schema::dropIfExists('transaction_items'); }  
    };
```

2. Enums

app/Modules/Sales/Enums/PaymentType.php

```
<?php  
  
namespace App\Modules\Sales\Enums;  
  
enum PaymentType: string  
{  
    case Cash      = 'cash';  
    case Transfer  = 'transfer';  
    case EWallet   = 'ewallet';  
    case EDC       = 'edc';  
    case QRIS      = 'qris';  
    case Credit    = 'credit'; // Piutang  
  
    public function label(): string  
    {  
        return match ($this) {  
            self::Cash      => 'Tunai',  
            self::Transfer  => 'Transfer Bank',  
            self::EWallet   => 'E-Wallet',  
            self::EDC       => 'Kartu (EDC)',  
            self::QRIS      => 'QRIS',  
            self::Credit    => 'Piutang',  
        };  
    }  
}
```

app/Modules/Sales/Enums/ShiftStatus.php

```
<?php

namespace App\Modules\Sales\Enums;

enum ShiftStatus: string
{
    case Open    = 'open';
    case Closed  = 'closed';

    public function label(): string
    {
        return match ($this) {
            self::Open    => 'Buka',
            self::Closed => 'Tutup',
        };
    }
}
```

app/Modules/Sales/Enums/TransactionStatus.php

```
<?php

namespace App\Modules\Sales\Enums;

enum TransactionStatus: string
{
    case Pending    = 'pending';
    case Completed  = 'completed';
    case Refunded   = 'refunded';
    case Cancelled  = 'cancelled';

    public function label(): string
    {
        return match ($this) {
            self::Pending    => 'Tertunda',
            self::Completed  => 'Selesai',
            self::Refunded   => 'Dikembalikan (Refund)',
            self::Cancelled  => 'Dibatalkan',
        };
    }
}
```

3. Models

app/Modules/Sales/Models/PaymentMethod.php

```
<?php
```

```
namespace App\Modules\Sales\Models;
```

```
use App\Modules\Sales\Enums\PaymentType;
```

```
use App\Shared\Traits\BelongsToBusiness;
```

```
use App\Shared\Traits\HasUuid;
```

```
use Illuminate\Database\Eloquent\Model;
```

```
use Illuminate\Database\Eloquent\SoftDeletes;
```

```
class PaymentMethod extends Model
```

```
{
```

```
    use HasUuid, BelongsToBusiness, SoftDeletes;
```

```
    protected $fillable = [
```

```
        'id', 'business_id', 'name', 'type',
```

```
        'provider', 'account_number', 'is_active', 'sort_order',
```

```
    ];
```

```
    protected function casts(): array
```

```
{
```

```
        return [
```

```
            'type' => PaymentType::class,
```

```
            'is_active' => 'boolean',
```

```
        ];
```

```
    }
```

```
    public function scopeActive($query)
```

```
{
```

```
        return $query->where('is_active', true)->orderBy('sort_order');
```

```
    }
```

```
}
```

app/Modules/Sales/Models/Shift.php

```
<?php
```

```
namespace App\Modules\Sales\Models;
```

```
use App\Models\User;
```

```
use App\Modules\Sales\Enums\ShiftStatus;
```

```
use App\Modules\Tenant\Models\Branch;
```

```
use App\Shared\Traits\BelongsToBusiness;
```

```
use App\Shared\Traits\HasUuid;
```

```
use Illuminate\Database\Eloquent\Model;
```

```
use Illuminate\Database\Eloquent\Relations\BelongsTo;
```

```
class Shift extends Model
```

```
{
    use HasUuid, BelongsToBusiness;

    protected $fillable = [
        'id', 'business_id', 'branch_id', 'user_id',
        'opened_at', 'closed_at',
        'starting_cash', 'actual_ending_cash', 'expected_ending_cash', 'difference',
        'status', 'notes',
    ];

    protected function casts(): array
    {
        return [
            'status'          => ShiftStatus::class,
            'opened_at'       => 'datetime',
            'closed_at'       => 'datetime',
            'starting_cash'   => 'integer',
            'actual_ending_cash' => 'integer',
            'expected_ending_cash' => 'integer',
            'difference'      => 'integer',
        ];
    }

    public function branch(): BelongsTo
    {
        return $this->belongsTo(Branch::class, 'branch_id');
    }
}
```

```
public function user(): BelongsTo
{
    return $this->belongsTo(User::class, 'user_id');
}

public function transactions(): \Illuminate\Database\Eloquent\Relations\HasMany
{
    return $this->hasMany(Transaction::class, 'shift_id');
}

public function isOpen(): bool
{
    return $this->status === ShiftStatus::Open;
}
}
```

app/Modules/Sales/Models/Transaction.php

```
<?php
```

```
namespace App\Modules\Sales\Models;
```

```
use App\Models\User;
```

```
use App\Modules\Sales\Enums\TransactionStatus;
```

```
use App\Modules\Tenant\Models\Branch;
```

```
use App\Shared\Helpers\CodeGenerator;
```

```
use App\Shared\Traits\BelongsToBusiness;
```

```
use App\Shared\Traits\HasUuid;
```

```
use Illuminate\Database\Eloquent\Model;
```

```
use Illuminate\Database\Eloquent\Relations\BelongsTo;
```

```
use Illuminate\Database\Eloquent\Relations\HasMany;
```

```
class Transaction extends Model
```

```
{
```

```
    use HasUuid, BelongsToBusiness;
```

```
    protected $fillable = [
```

```
        'id', 'business_id', 'branch_id', 'user_id', 'shift_id',
```

```
        'customer_id', 'payment_method_id',
```

```
        'invoice_number', 'status', 'payment_status', 'order_type',
```

```
        'subtotal', 'discount_amount', 'tax_amount', 'service_charge', 'grand_total',
```

```
        'amount_paid', 'change_amount', 'currency',
```

```
        'payment_reference', 'notes',
```

```
    ];
```

```
    protected function casts(): array
```

```
{
```

```
        return [
```

```
            'status' => TransactionStatus::class,
```

```
            'subtotal' => 'integer',
```

```
            'discount_amount' => 'integer',
```

```
            'tax_amount' => 'integer',
```

```
            'service_charge' => 'integer',
```

```
            'grand_total' => 'integer',
```

```
            'amount_paid' => 'integer',
```

```
            'change_amount' => 'integer',
```

```
        ];
```

```
    }
```



```

protected static function boot(): void
{
    parent::boot();

    static::creating(function (self $model) {
        if (empty($model->invoice_number)) {
            $model->invoice_number = CodeGenerator::invoiceNumber('INV');
        }
    });
}

/* — Relations — */

public function branch(): BelongsTo
{
    return $this->belongsTo(Branch::class, 'branch_id');
}

public function user(): BelongsTo
{
    return $this->belongsTo(User::class, 'user_id');
}

public function shift(): BelongsTo
{
    return $this->belongsTo(Shift::class, 'shift_id');
}

public function paymentMethod(): BelongsTo
{
    return $this->belongsTo(PaymentMethod::class, 'payment_method_id');
}

public function items(): HasMany
{
    return $this->hasMany(TransactionItem::class, 'transaction_id');
}

/* — Helpers — */

public function isCompleted(): bool
{
    return $this->status === TransactionStatus::Completed;
}

```

}

}

app/Modules/Sales/Models/TransactionItem.php

```
<?php

namespace App\Modules\Sales\Models;

use App\Modules\POS\Models\Product;
use App\Modules\POS\Models\ProductVariant;
use App\Shared\Traits\HasUuid;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class TransactionItem extends Model
{
    use HasUuid;

    protected $fillable = [
        'id', 'transaction_id', 'product_id', 'product_variant_id',
        'product_name', 'variant_name', 'sku',
        'quantity', 'unit_price', 'cost_price',
        'discount_amount', 'tax_amount', 'subtotal', 'notes',
    ];

    protected function casts(): array
    {
        return [
            'quantity'      => 'integer',
            'unit_price'     => 'integer',
            'cost_price'     => 'integer',
            'discount_amount' => 'integer',
            'tax_amount'     => 'integer',
            'subtotal'      => 'integer',
        ];
    }

    /* — Relations — */

    public function transaction(): BelongsTo
    {
        return $this->belongsTo(Transaction::class, 'transaction_id');
    }

    public function product(): BelongsTo
```

```
{  
    return $this->belongsTo(Product::class, 'product_id');  
}  
  
public function variant(): BelongsTo  
{  
    return $this->belongsTo(ProductVariant::class, 'product_variant_id');  
}  
}
```